Backend review · for Abhijeet

Attainment engine backend: review and fix list

Reviewed: `attainment-engine (2).zip` → `backend/` (Spring Boot 4.1.1, Java 21; 44 Java files, about 3,000 lines; every file read) · Checked against `frontend/API_CONTRACT.md` v2.5 · 28 Oct 2026 · From Rohan

Summary. The code compiles, and the new **Faculty** and **Faculty-course assignment** modules are well built: request and response DTOs, `@Valid`, proper duplicate checks. It isn't ready to merge yet. **127 tests** were written for this review: **100 pass** and **27 fail**. Each failure is a confirmed defect, listed below with where it is and how to fix it. The five original modules are unchanged since the first review, so the earlier issues are still open. One security item needs action today.

127	100	27	1
tests written	pass	fail (= defects)	leaked credential

Do this first: change the MongoDB Atlas password. `.idea/workspace.xml` and `.idea-old/workspace.xml` in the zip contain the full connection string, password included, for user `copo_app_user` on `copo-dev.mm2mhwg.mongodb.net` (IntelliJ saves run-configuration environment variables there). The zip has been shared, so treat the password as exposed.

- In Atlas → **Database Access**, edit `copo_app_user` and set a new password. Update your local run configuration.
- Never zip or commit `.idea/`, `.idea-old/` or `target/`. Share code through Git, where `backend/.gitignore` already excludes them.
- Good news: it has never reached our GitHub repo (full history checked), and `application.properties` correctly reads `${MONGODB_URI}`.

How this was tested

- **Build:** `./mvnw compile` succeeds. Your own test (`contextLoads`) **fails without a database**: startup needs `MONGODB_URI`, and `MongoConnectionTest` pings Atlas on every boot.
- **Service tests** for all 9 modules, with repositories mocked through Mockito, so no database is needed. They cover creation, duplicates, not-found, status rules, scope and year rules, and update behaviour.
- **HTTP tests** through Spring MVC (`MockMvcBuilders.standaloneSetup`): bean validation, status codes, query parameters and request binding.
- **Convention checks** against the contract: `/api/v1` prefix, global error handler, CORS, entities as request bodies, startup ping, index creation, package/folder match, and secrets in properties.
- Tests named `[BUG] …` state what the contract or common sense requires. They **fail today**, and each turns green when its defect is fixed. Every other test passes.
- Not covered: behaviour against a real MongoDB (Docker wasn't running). That would also show whether the unique indexes actually exist.

What's already good

- GOOD **Faculty:** `FacultyRequest` / `FacultyResponse` DTOs, `@Valid`, email lower-cased, duplicate code and email blocked, and updates can't move an institution or take another member's code.
- GOOD **Assignments:** DTOs, the academic-year format is checked, faculty and course must exist and be ACTIVE, and duplicates return 409.
- GOOD **Semester:** year *n* accepts exactly semesters 2n−1 and 2n; every other combination returns 400 (tested for all 4 years, plus 0, 9 and negatives).
- GOOD **Course:** a DTO with `@Valid`; codes trimmed and upper-cased; the semester must exist and be ACTIVE.
- GOOD **Program year and Syllabus:** all scope rules correct; a `DuplicateKeyException` race becomes 409.
- GOOD No secrets in `application.properties`; port 8081 and the app name are set.

Fix list, in order

Priority: P1 data safety and contract plumbing (everything else builds on these) · P2 module bugs · P3 tidy-up. The grey line under each item names the test that goes green when it's fixed.

P1-1 A POST can overwrite an existing record (mass assignment)

11 endpoints bind the MongoDB `@Document` entity straight from the request body. A body that includes `"id"` of an existing record makes `save()` replace it, and also lets the client set `status`, `createdAt` and so on. This was proven over HTTP on `POST /api/departments`.

Affected: `Department`, `Institution`, `Program`, `ProgramYear` and `Syllabus` controllers (create and update) and `SemesterController.create`.

Fix

Do what you already did for Course and Faculty: one request DTO per create/update, with validation, mapped onto a *new* entity in the service.

```
public record DepartmentRequest(
    @NotBlank String institutionId,
    @NotBlank @Size(max = 20) String departmentCode,
    @NotBlank String departmentName) {}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public DepartmentResponse create(@Valid @RequestBody DepartmentRequest req) { ... }
```

Tests: noEntityRequestBodies · departmentPostIgnoresClientId · createIgnoresClientId · emptyDepartmentIs400

P1-2 No global error handler, so no contract error format

There is no `@RestControllerAdvice`. Errors come back in Spring's default shape (`status`, `error`, `path`) with no `errorCode`, and successes return raw objects. The contract (§2.2, §18) expects an `ApiResponse<T>` envelope, and the frontend reads `errorCode` and `fieldErrors`.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(MethodArgumentNotValidException.class)
  ResponseEntity<ApiError> invalid(MethodArgumentNotValidException ex) {
    Map<String,String> fields = new LinkedHashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(e -> fields.put(e.getField(), e.getDefaultMessage()));
    return ResponseEntity.badRequest().body(ApiError.of("VALIDATION_FAILED", "Check the highlighted
fields.", fields));
  }
  @ExceptionHandler(ResponseStatusException.class)       // 404 NOT_FOUND, 409 DUPLICATE_*, 400 ...
  @ExceptionHandler(DuplicateKeyException.class)         // -> 409
  @ExceptionHandler(HttpMessageNotReadableException.class)// bad JSON -> 400
  @ExceptionHandler(Exception.class)                     // -> 500 INTERNAL_ERROR, log it, generic message
}
```

Return `ApiResponse.ok(data)` (`{ success, data, message?, timestamp }`) from every controller.

Tests: globalErrorHandlerExists

P1-3 Routes aren't under `/api/v1`

7 of 9 controllers use `/api/...`: courses, departments, institutions, program-years, programs, semesters and syllabi. Faculty and faculty-course-assignments already use `/api/v1/...`. The frontend calls `http://localhost:8080/api/v1`.

Fix

Change the seven `@RequestMapping`s to `/api/v1/...`, and agree on a port with Rohan: today the backend is on 8081 and the frontend expects 8080.

Test: allRoutesVersioned

P1-4 No CORS, so the Angular app can't call the API

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
  @Value("${app.cors.allowed-origins:http://localhost:4200}") String[] origins;
  public void addCorsMappings(CorsRegistry r) {
    r.addMapping("/api/**").allowedOrigins(origins).allowedMethods("GET","POST","PUT","PATCH","DELETE");
  }
}
```

Test: corsConfigured

P1-5 Unique indexes are declared but never created

`@Indexed(unique=true)` and `@CompoundIndex(unique=true)` exist on Institution, ProgramYear, Course, Semester and FacultyCourseAssignment. But Spring Data MongoDB doesn't create indexes unless told to, so two simultaneous requests can both pass the `existsBy…` check and insert duplicates.

Fix

- Add `spring.data.mongodb.auto-index-creation=true`, or create the indexes in a startup migration.
- Add the missing unique indexes on Department (`institutionId+departmentCode`), Program (`departmentId+programCode`) and Faculty (`institutionId+facultyCode`, `institutionId+email`).
- Check in Atlas with `db.courses.getIndexes()`.

`Test: indexCreationEnabled`

P1-6 The app can't start, or be tested, without Atlas

`MongoConnectionTest.java:14` is a `CommandLineRunner` that pings Atlas on every boot, and `contextLoads` needs a live `MONGODB_URI`, so the build fails for anyone without your credentials.

Fix

Delete `MongoConnectionTest`; Spring Boot's health check already covers the connection. For the context test, use Testcontainers (`@ServiceConnection MongoDBContainer`) or move it to an `integration` profile.

`Test: noStartupPing (and your contextLoads)`

Module bugs

#	Module	Defect (where)	Fix · test
P2	Institution	Unknown id throws `RuntimeException` → **500** (`InstitutionService.java:41, 46, 58, 68`)	Throw `ResponseStatusException(NOT_FOUND)` like the other services · `unknownIdIs404`
P2	Institution	New institution saved with **no status** (`:33`)	`setStatus("ACTIVE")` before save · `newInstitutionIsActive`
P2	Institution	Update copies `status` from the client, bypassing deactivate/reactivate (`:50`)	Remove that line · `updateIgnoresStatus`
P2	Institution	Update can take another institution's code (`:48`)	Check `existsByInstitutionCode` when the code changes, else 409 · `updateToDuplicateCodeIs409`
P2	Department	No validation: `POST {}` → 200 with an empty record; a missing code is saved as `null`	DTO + `@NotBlank` (P1-1) · `missingCodeIs400` · `emptyDepartmentIs400`
P2	Department	Update without a code **erases** the stored code (`DepartmentService.java:76`)	Validate the update DTO; never copy nulls · `updateWithoutCodeKeepsCode`
P2	Department	Codes not normalised: `" ece "` and `ECE` are different	`trim().toUpperCase()`, as Course does · `codeIsNormalised`

P2	Department	Can be created under an **INACTIVE** institution (`:28` only checks `existsById`)	Load it and require `ACTIVE` · `inactiveInstitutionRejected`
P2	Program	`programLevel` accepts any text (`ProgramService.java:79`)	Enum `UG | PG` (contract §6) · `invalidLevelRejected`
P2	Course	`courseType` accepts any text, e.g. "banana" (`CourseService.java:35`)	Enum with the contract's `CourseType` values (CORE, PROGRAM_ELECTIVE, OPEN_ELECTIVE, …) · `unknownCourseTypeRejected`
P2	Course	A duplicate caught by the index throws a raw `DuplicateKeyException` → 500 (`:66`)	Catch it → 409, as ProgramYear does · `raceIs409`
P2	Semester	Can be added to an **INACTIVE** program year (`SemesterController.java:43`)	Require `ACTIVE` · `inactiveYearRejected`
P2	Semester	Listing for an unknown program year returns `[]` instead of 404 (`:83`)	Check `existsById` first · `listUnknownYearIs404`
P2	Faculty	Codes not upper-cased: `f001` and `F001` both get in (`FacultyService.java:36, 100`)	`trim().toUpperCase()` · `codeCaseNormalised`
P2	Faculty	Can join an **INACTIVE** department (`validateDepartment`)	Require `ACTIVE` · `inactiveDepartmentRejected`
P2	Assignment	Academic year `2026-99` accepted (`FacultyCourseAssignmentRequest.java:17`)	Also check that the second part is (first year + 1) mod 100 · `nonConsecutiveYearRejected`

Tidy-up

#	Issue	Fix · test
P3	`FacultyCourseAssignment.java` and `FacultyCourseAssignmentRepository.java` sit in `academic/model` and `academic/repository` but declare `faculty.*` packages	Move them to `faculty/model` and `faculty/repository` · `packagesMatchFolders`
P3	`SemesterController` holds the business logic and talks to repositories directly	Move it into a `SemesterService` , like the other modules
P3	`POST` returns **200** for Department and Institution but 201 elsewhere	`@ResponseStatus(HttpStatus.CREATED)` on every create
P3	Scope is case-sensitive in `SyllabusService.java:62` but not in `ProgramYearService.java:52`	Use a `SyllabusScope` enum, and the same for `status` (`ACTIVE | INACTIVE`)
P3	Leftovers: `System.out.print` in `DepartmentController` ; Hinglish comments in `InstitutionController` ; empty `static/` and `templates/` ; devtools inside `<dependencyManagement>` (no effect); unused `spring-ai-bom`	Remove, or use SLF4J for logging

Contract alignment

These are design points to settle with Rohan rather than bugs, because the contract will follow whatever you agree:

Topic	Contract today	Your code
ID field names	`departmentId`, `programId`, …	`id` everywhere. Use the contract names in response DTOs
Semesters	Belong to a syllabus (ACAD-6), with a `studyYear`	Belong to a `ProgramYear`. Either is fine; agree on one
Course	A course master (`courseCode`, `courseName`, `courseType`, ADM-7) plus a per-semester syllabus course (ADM-8)	One `Course` per semester holds both. Fine for now; note the difference
Subject allocation	§15.2: the coordinator allocates, with a course-level role (theory, practical, coordinator…)	Assignment has no role and no "who allocated". Add them before the faculty dashboard uses it
Institution	A single institution (ADM-1)	Multi-institution CRUD. Decide together

Using the tests

1. Copy the three files sent with this report into `backend/src/test/java/com/copo/attainment_engine/review/`: `AcademicStructureServiceTest.java`, `SemesterCourseFacultyTest.java`, `HttpAndConventionsTest.java`.
2. Run: `./mvnw test -Dtest="com.copo.attainment_engine.review.**"`. They need no database.
3. Today: 100 pass, 27 fail. Fix in the order above and re-run; the goal is **127 / 127**.
4. When you switch an endpoint to a DTO (P1-1), update the matching HTTP test to the new signature. The intent stays the same: a client can't choose the id.

Generated from a full read of the zip and test runs on a clean working copy (IDE folders and credentials excluded). Line numbers refer to the files in `attainment-engine (2).zip`. Questions: Rohan.